

---

# COMBINING MARKS MATTER: PRE-TOKENISATION HALVES TELUGU TOKEN COUNTS AT A ONE-PERCENT COST TO LATIN SCRIPTS

---

Nikhil Raj Mule  
nikhilrajmule@gmail.com

September 12, 2026

## ABSTRACT

The regular expression that splits text before byte-pair encoding is rarely examined, yet it determines what BPE can learn. We show that the pattern used by `cl100k_base`, the tokeniser behind GPT-3.5 and GPT-4, matches letters with `\p{L}+` and therefore excludes Unicode combining marks. In Telugu, an abugida in which vowel signs are combining marks, this fragments every word before merging begins. We quantify the cost with a controlled comparison: two byte-level BPE tokenisers, identical vocabulary size, corpus, sample and trainer, differing only in this character class. On a 25.9 GB Telugu corpus the corrected pattern reduces token count by 51.8% and moves the cross-language compression ratio from 2.35 to 1.10, close to parity with English and German. We then pre-train matched 533M-parameter decoder models on an identical 9B-token budget under each tokeniser. Held-out bits-per-byte improves by 5.5% for Telugu at a cost of roughly 1% for English and German, a trade-off whose direction the compression statistics predict. Its magnitude they do not: halving the token count buys only a 5.5% reduction in bits-per-byte, so the benefit accrues mainly to inference cost and effective context length rather than to model quality. After identical supervised fine-tuning, extractive question answering moves in the same direction, with Telugu exact match improving 45% relative. We report two negative results and release all code, tokenisers and synthetic evaluation data.

## 1 Introduction

Tokenisation sits upstream of everything a language model does. It fixes the units the model can represent, the amount of text a context window holds, and the cost of serving a given document. When a tokeniser represents one language less efficiently than another, every downstream system inherits that asymmetry as higher inference cost, shorter effective context, and less content seen per unit of training compute [1, 2].

Most attention to this problem has focused on vocabulary allocation: how many merges a language receives during BPE training. We identify a different and earlier failure. Before BPE runs, text is split by a regular expression into chunks across which merges may not cross. The pattern used by `cl100k_base` matches letter runs with `\p{L}+`, a class that excludes Unicode combining marks (`\p{M}`). Telugu, like other Brahmic abugidas, writes vowels as combining marks attached to consonants. A Telugu word is therefore shattered into fragments *before* the BPE algorithm sees it, and no amount of vocabulary budget can undo the split.

Figure 1 shows what this does to a single sentence. Under the `cl100k` pattern the four-word Telugu sentence “*hāy, nēnu parikṣistūnnānu.*” is cut into fifteen chunks. The cuts do not fall at word boundaries; they fall inside orthographic syllables, separating vowel signs from the consonants they modify, and several of the resulting chunks are not well-formed Telugu at all. The `o200k` pattern produces five chunks, corresponding exactly to the sentence’s words and punctuation. Since BPE can only merge within a chunk, the top row of the figure is an upper bound on what any amount of subsequent merging can recover.

The fix already exists, but it is not documented. In `o200k_base`, the tokeniser for GPT-4o, OpenAI added `\p{M}` to the letter classes. The change is visible in the public `tiktoken` source, where the accompanying comment notes that token

## Pre-tokenisation of a Telugu sentence

c100k pattern  $(\backslash p\{L\}+)$   $\rightarrow$  15 pieces

హాయ్, నేను పరీక్షిస్తున్నాను.

o200k pattern  $(+\backslash p\{M\})$   $\rightarrow$  5 pieces

హాయ్, నేను పరీక్షిస్తున్నాను.

Figure 1: Pre-tokenisation of the Telugu sentence “*hāy, nēnu parīkṣistūnnānu.*” (“Hi, I am testing.”). Each box is one chunk produced by the regular expression; BPE may merge within a box but never across boxes. The c100k pattern matches letters with  $\backslash p\{L\}+$ , which excludes combining marks, and produces 15 chunks in which vowel signs are separated from the consonants they modify. The o200k pattern includes  $\backslash p\{M\}$  and produces 5 chunks corresponding to the words and punctuation of the sentence.

allocation across languages could be done more efficiently [3], but we are not aware of any published measurement of what the change is worth.

This paper supplies that measurement. Our contributions are:

- We isolate pre-tokenisation as the mechanism and quantify its cost on a 25.9 GB Telugu corpus: a 51.8% reduction in token count at identical vocabulary size, and a cross-language compression ratio falling from 2.35 to 1.10.
- We run a controlled downstream comparison. Two 533M-parameter models are pre-trained on an identical 9B-token budget, identical architecture, and identical data mixture, differing only in tokeniser. Telugu bits-per-byte improves by 5.5%; English and German degrade by about 1%.
- After identical supervised fine-tuning, Telugu extractive question answering improves by 45% relative in exact match, agreeing in direction with the bits-per-byte result.
- We show that compression gains do not translate proportionally into modelling gains, and argue that tokeniser selection for a low-resource language is therefore primarily an economic decision.
- We report two negative results: an additional character-class change with no measurable effect, and codepoint initialisation, whose benefit we show to be a small-vocabulary artefact.
- We release the tokenisers, the full pipeline, and a synthetic German invoice QA dataset.

## 2 Related Work

**Subword tokenisation.** Byte-pair encoding was introduced as a compression algorithm [4] and adapted to open-vocabulary machine translation [5]. GPT-2 established the byte-level variant now standard in decoder models, in which the base alphabet is the 256 byte values and merges are learned above them, guaranteeing that any input is representable [6]. Regular-expression pre-tokenisation was introduced in the same work and has been carried forward largely unchanged.

**Tokeniser fairness.** Several studies document that tokenisers trained predominantly on English represent other languages less efficiently, with direct consequences for API cost and context length [1, 2], and that tokenisation

Table 1: Corpora after cleaning.

|                       | English   | German    | Telugu    |
|-----------------------|-----------|-----------|-----------|
| Documents read        | 4,431,410 | 6,716,695 | 4,346,994 |
| dropped, length       | 3,814     | 3,694     | 504       |
| dropped, exact dup.   | 87,947    | 24        | 9,969     |
| dropped, wrong script | 6,934     | 1,377     | 114,803   |
| dropped, near dup.    | 49,408    | 26,702    | 214,035   |
| Documents kept        | 4,283,307 | 6,684,898 | 4,007,683 |
| Retention             | 96.7%     | 99.5%     | 92.2%     |
| Size                  | 19.65 GB  | 20.24 GB  | 25.89 GB  |

choices affect multilingual model quality [7]. This literature generally treats the tokeniser as a single object and varies vocabulary size or training corpus. We hold both fixed and vary only the pre-tokenisation pattern.

**Indic language resources.** AI4Bharat’s Sangraha corpus and IndicQA benchmark provide the Telugu data used here [8, 9], building on earlier IndicNLP resources [10]. Our English and German corpora come from FineWeb and FineWeb-2 [11, 12].

**Training recipes.** Our training code follows nanoGPT [13]. We adopt bits-per-byte as the primary metric following nanochat, which uses it because it is comparable across tokenisers and low-variance relative to task benchmarks [14]. Model and data sizes follow compute-optimal scaling [15].

### 3 Data

**Sources.** Telugu is drawn from FineWeb-2 (`tel_Telu`), Sangraha (`verified/tel`), CulturaX [16] and IndicCorpV2 [17]. German is FineWeb-2 (`deu_Latn`) and English is FineWeb-Edu (`sample-10BT`). Telugu required four sources to reach a usable size; English and German each required one. That asymmetry is itself a measure of the resource gap this paper is about.

**Cleaning.** Documents are NFC-normalised, then filtered in four stages: length (200–100,000 characters), exact duplicate detection by document hash, script verification, and near-duplicate detection. Script verification requires that at least 70% of alphabetic characters fall in the Telugu Unicode block (U+0C00–U+0C7F). For English and German we checked a sample of 500 documents and found FineWeb-2’s own language identification correct in 499 cases, so we did not filter further. Near-duplicate detection uses MinHash LSH over word 5-grams with 32 hashes in 8 bands of 4, treating a document as duplicate when it matches on two or more bands.

Table 1 reports the outcome. Two asymmetries in it are worth noting. Telugu loses 114,803 documents (2.6%) to script verification against 0.16% for English and 0.02% for German, which tells us the Telugu sources carry substantially more mislabelled and romanised text. In the other direction, German loses only 24 documents to exact duplication against English’s 87,947, suggesting FineWeb-2’s German pipeline deduplicates far more aggressively upstream than FineWeb-Edu’s English one. Within Telugu, near-duplicate detection removes 21 times more documents than exact hashing (214,035 against 9,969), which is what justifies the cost of the MinHash stage.

**Held-out data.** Every 37th document of each cleaned corpus is reserved, up to 50 MB per language. We do this *before* tokeniser training, so no held-out text influences the merge tables, and we sample throughout the corpus rather than taking the first 50 MB, which would reflect a single region of a single crawl.

### 4 Tokenisers

Both tokenisers are byte-level BPE with a 48,000-token vocabulary (47,744 learned merges above the 256 byte values), trained on the same 350 MB byte-balanced sample drawn uniformly at random from the training corpora, using the same trainer with deterministic tie-breaking. The only difference is the pre-tokenisation pattern: `c1100k_pat` reproduces the `c1100k_base` regex, `o200k_pat` reproduces the `o200k_base` regex, which differs in including `\p{M}` in its letter classes.

Table 2: Compression on held-out text. Characters per token is the cross-script comparable measure; bytes per token is reported because bits-per-byte normalises by bytes. Ratio is worst language over best.

| Pattern    | characters / token |      |      |             | bytes / token |      |       |
|------------|--------------------|------|------|-------------|---------------|------|-------|
|            | EN                 | DE   | TE   | Ratio       | EN            | DE   | TE    |
| c1100k_pat | 4.52               | 4.65 | 1.98 | 2.35        | 4.54          | 4.72 | 5.26  |
| o200k_pat  | 4.43               | 4.53 | 4.13 | <b>1.10</b> | 4.46          | 4.60 | 10.98 |

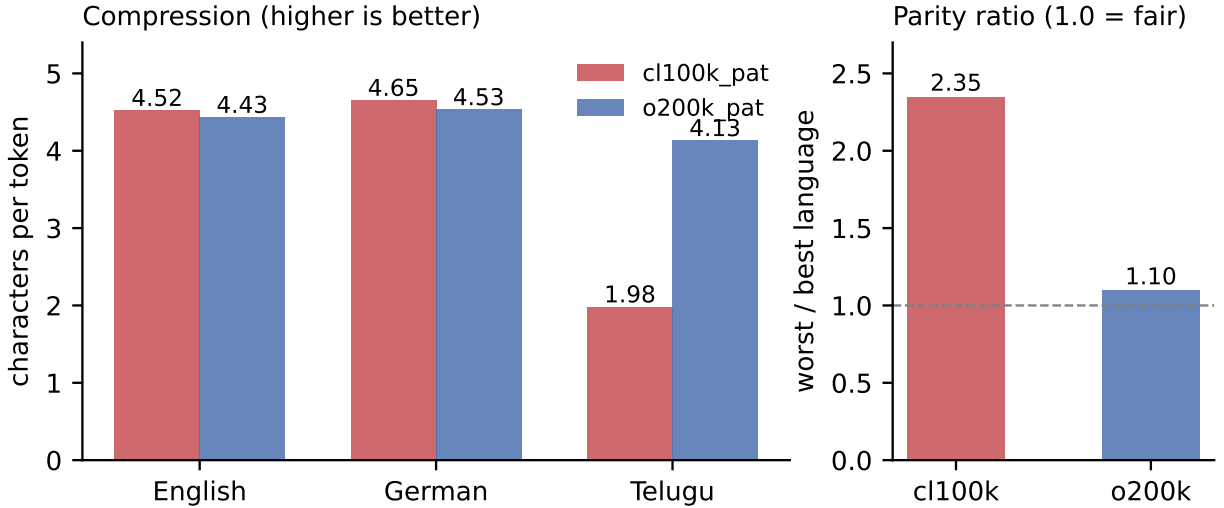


Figure 2: Left: characters per token on held-out text; higher means the same text costs fewer tokens. Right: the parity ratio, worst language over best, where 1.0 would mean all three languages are served equally. The o200k pattern moves Telugu from 1.98 to 4.13 characters per token and the ratio from 2.35 to 1.10, at a cost of roughly 2% on English and German.

The two patterns see different numbers of word types in the same sample: 2,454,619 for c1100k\_pat against 3,698,054 for o200k\_pat. The larger count follows directly from the mechanism. Keeping Telugu words whole produces many more distinct types than cutting them into a small set of recurring fragments.

Table 2 and Figure 2 show the effect. Telugu compression more than doubles, from 1.98 to 4.13 characters per token, while English and German move about 2% in the opposite direction. The right panel of Figure 2 plots the consequence for cross-language parity: the ratio of the worst-served language to the best falls from 2.35 to 1.10. Put in user terms, under c1100k\_pat a Telugu speaker is charged more than twice as many tokens as an English speaker for the same amount of text, and under o200k\_pat the three languages sit within 10% of one another.

One caution on reading Table 2: characters per token, not bytes per token, is the comparable measure across scripts. Telugu codepoints occupy three UTF-8 bytes each, so the bytes-per-token column flatters Telugu and should not be read as showing it better served than English. We report bytes per token only because bits-per-byte normalises by bytes.

At corpus scale the same text tokenises to 4,932.5M tokens under c1100k\_pat and 2,375.2M under o200k\_pat, a **51.8% reduction**, shown in Figure 3. English and German move the other way by about 2%, which is the cost of the change rather than a measurement artefact. We measured the Telugu reduction three times — on a 2 MB probe, on an earlier 8.95 GB corpus, and on the final 25.89 GB corpus, using two different sampling schemes for the tokeniser training set — obtaining 52.0%, 52.3% and 52.1%. The figure is not an artefact of one corpus or one sample.

## 5 Experimental Setup

**Model.** A 533.44M-parameter decoder: 24 layers, model dimension 1280, 20 attention heads, context length 1024, no biases, tied input and output embeddings, vocabulary padded from 48,001 to 48,064 for tensor alignment.

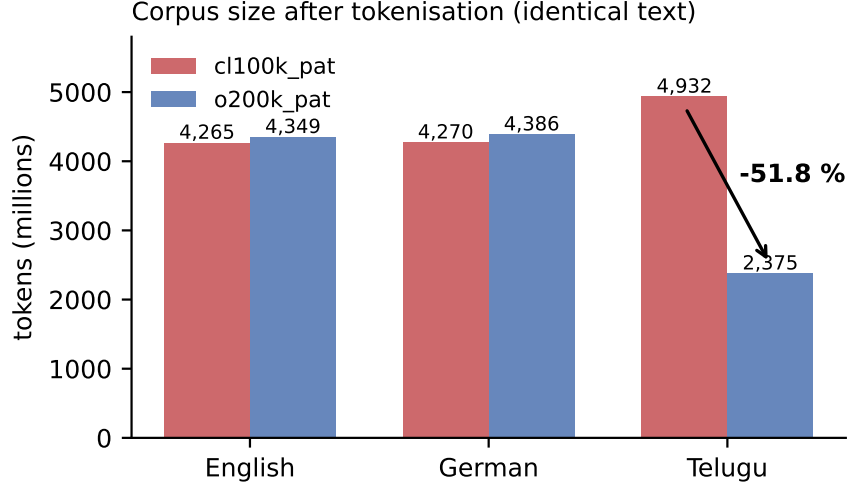


Figure 3: Token counts for the identical cleaned corpus under each pattern. The text is the same in both bars of each pair; only the tokeniser differs. Telugu falls from 4,932.5M to 2,375.2M tokens, a 51.8% reduction, while English and German rise by about 2%.

**Data budget.** Each model is trained on 9B tokens mixed 40/35/25 across English, German and Telugu *by token count*, interleaved in 1024-token blocks with a fixed shuffle seed. Under both tokenisers every language stays below one epoch (Telugu: 0.95 for o200k\_pat, 0.46 for cl100k\_pat), so no language is repeated and no repetition asymmetry is introduced.

We hold the *token* budget fixed rather than the text budget. The o200k\_pat model consequently sees roughly twice as much Telugu text within its 25% share. This is the mechanism under test rather than a confound: the token budget is what determines training cost, and a tokeniser that fits more content into the same budget is doing exactly what a tokeniser is for.

**Optimisation.** AdamW, peak learning rate  $3 \times 10^{-4}$  with cosine decay to  $3 \times 10^{-5}$ , 340 warmup steps, gradient clipping at 1.0, bfloat16. Micro-batch 8 with 64 gradient accumulation steps gives 524,288 tokens per iteration; 17,000 iterations complete the 9B-token budget.

**Hardware.** A single NVIDIA RTX 5090. Throughput was 12.2 s per iteration at 49.2% model FLOPs utilisation, giving 57.6 hours per run. Throughput was stable across runs, so the comparison is not confounded by hardware variation.

**Seeds.** Two seeds (1337, 1338) per tokeniser, four pre-training runs in total. With  $n = 2$  we report the observed range rather than a standard deviation.

Figure 4 shows the pre-training loss for both o200k\_pat seeds. The curves are visually indistinguishable throughout and their final validation losses differ by 0.014, which bounds run-to-run variation at this scale. We deliberately plot a single tokeniser. Training loss is measured per token, the two tokenisers produce different tokens, and so their curves are not comparable: a tokeniser that splits text into many individually predictable fragments attains a lower per-token loss while representing the same text worse. The valid cross-tokeniser comparison is bits-per-byte, reported in Section 6.1.

## 6 Results

### 6.1 Bits per byte

Per-token loss is not comparable across tokenisers, since a tokeniser that produces more, individually easier tokens can achieve lower per-token loss while representing the same text worse. We therefore use bits-per-byte, which normalises the model’s total negative log-likelihood over a text by that text’s byte length — identical for both models. We evaluate on the held-out set with a strided window (block 1024, stride 512) so that every token is scored exactly once with at least 512 tokens of context.

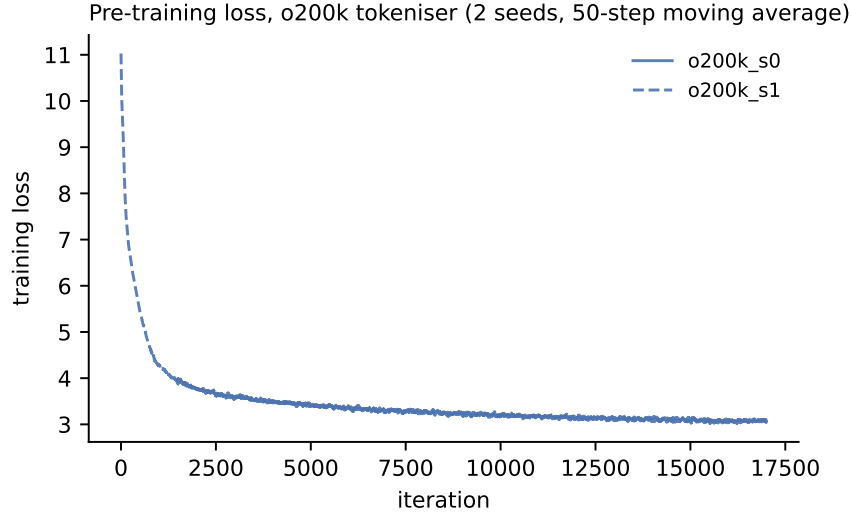


Figure 4: Pre-training loss for two o200k\_pat seeds, 50-step moving average. The curves are indistinguishable, bounding run-to-run variation. Loss is not comparable *across* tokenisers, since different vocabularies imply different per-token difficulty; the valid cross-tokeniser comparison is bits-per-byte (Table 3).

Table 3: Held-out bits per byte (lower is better), means over two seeds per tokeniser.

| Language | c1100k_pat    | o200k_pat     | Change |
|----------|---------------|---------------|--------|
| English  | <b>0.9449</b> | 0.9534        | +0.9%  |
| German   | <b>0.9781</b> | 0.9879        | +1.0%  |
| Telugu   | 0.4472        | <b>0.4225</b> | −5.5%  |

Table 3 and Figure 5 report the result. The outcome is a clean trade-off, and its direction follows the compression statistics in both arms. c1100k\_pat compresses English and German about 2% better (Table 2) and correspondingly achieves about 1% lower bits-per-byte on those languages. o200k\_pat compresses Telugu 2.1 times better and achieves 5.5% lower bits-per-byte. The pre-tokenisation fix is therefore not free: it buys a substantial Telugu gain at a small but measurable cost to the Latin-script languages sharing the vocabulary.

Seed-to-seed variation bounds these claims. Across two seeds per tokeniser, held-out bits-per-byte varies by at most 0.0020 in any language and condition, against tokeniser gaps of 0.0085 (English), 0.0098 (German) and 0.0247 (Telugu). No effect reported here falls within the noise of a re-run.

**Compression does not translate proportionally to quality.** A 51.8% reduction in Telugu token count yields a 5.5% reduction in bits-per-byte. The two quantities measure different things. Token count determines how much text fits in a context window and how much an API call costs; bits-per-byte measures how well the model predicts the underlying bytes. A practitioner choosing a tokeniser for a low-resource language should therefore expect the benefit to be primarily economic — half the tokens per document, twice the effective context — rather than a comparable improvement in model quality. To our knowledge this ratio has not previously been measured under controlled conditions.

## 6.2 Downstream question answering

Both pre-trained models are fine-tuned on an identical supervised set of 24,000 extractive QA examples drawn in equal parts from SQuAD 2.0 [18], GermanQuAD [19], TyDi QA Telugu [20], and a synthetic German invoice corpus we generate and release. Examples are formatted as context, question and answer, with the loss computed only over answer tokens; unanswerable questions are marked with an explicit token and balanced to approximately 30% in every language. Fine-tuning runs for three epochs at a peak learning rate of  $1 \times 10^{-5}$  with cosine decay, micro-batch 4 with 8 gradient accumulation steps, and we evaluate the final epoch. Generation is greedy with a 48-token limit, one example at a time, over 400 held-out questions per language.

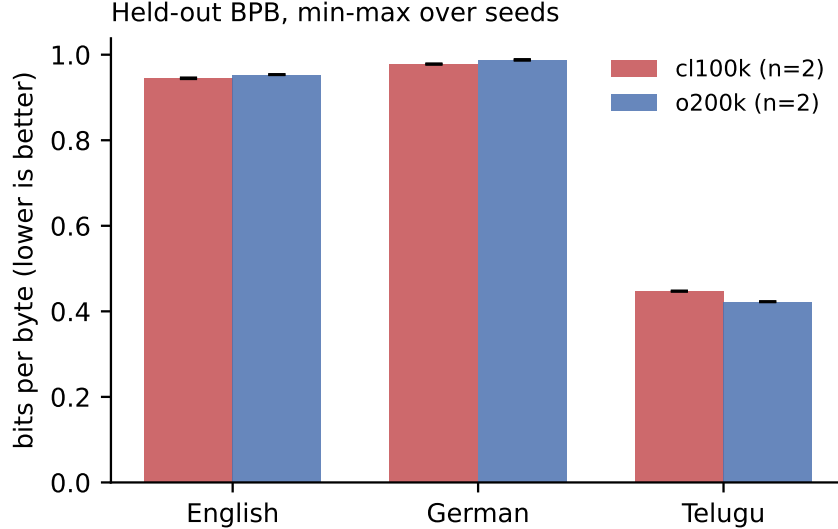


Figure 5: Held-out bits per byte, means over two seeds with min-max bars. Bars should be compared within a language only: scripts differ in byte-level entropy, so the Telugu bars being lower than the English ones says nothing about relative quality.

Table 4: Extractive QA after identical supervised fine-tuning, means over two seeds per tokeniser. HasEM and HasF1 are restricted to answerable questions; NoAcc is accuracy on unanswerable questions; span% is the fraction of predictions that are literal substrings of the context.

| Lang | Tokeniser  | HasEM       | HasF1       | NoAcc       | span%       |
|------|------------|-------------|-------------|-------------|-------------|
| EN   | cl100k_pat | <b>28.0</b> | <b>33.5</b> | 70.8        | 52.4        |
|      | o200k_pat  | 27.8        | 32.8        | <b>73.2</b> | 51.3        |
| DE   | cl100k_pat | 78.9        | 83.9        | <b>99.2</b> | <b>96.6</b> |
|      | o200k_pat  | <b>80.2</b> | <b>85.2</b> | 98.3        | 96.1        |
| TE   | cl100k_pat | 8.7         | 12.8        | <b>52.5</b> | 86.6        |
|      | o200k_pat  | <b>12.6</b> | <b>20.7</b> | 50.0        | <b>88.1</b> |

Table 4 reports the outcome. We take the answerable subset as the primary measure, because overall exact match is dominated by the unanswerable fraction, on which both models score highly, and therefore flatters both.

Telugu extraction improves by 45% relative in exact match (8.7 to 12.6) and 62% relative in F1 (12.8 to 20.7). English is unchanged and German improves slightly. The direction agrees with the bits-per-byte result and the magnitude is larger, which is what one would expect of a task that requires reproducing a contiguous span of the source text: a tokenisation that keeps Telugu words whole makes the span easier to emit exactly.

We do not lean on this evidence as heavily as on bits-per-byte. Telugu HasEM across individual runs is 7.7 and 9.7 for cl100k\_pat and 12.1 and 13.1 for o200k\_pat; the arms do not overlap, but the gap of 3.9 points exceeds the within-arm spread by roughly a factor of two, where the bits-per-byte gaps exceed their spread by a factor of ten or more. Absolute Telugu scores are also low in both arms, which reflects a small and partly translated fine-tuning set evaluated against a human-authored benchmark.

The span% column is worth reporting separately. German predictions are literal substrings of the context 96% of the time and Telugu 87–88%, but English only about 52%: on English the model frequently produces an answer that does not appear verbatim in the passage. We did not anticipate this asymmetry and do not have an explanation for it.

## 7 Negative Results

**Excluding combining marks from the punctuation class.** We hypothesised that also removing `\p{M}` from the negated punctuation class would further improve Telugu segmentation. The modified pattern does behave differently — it yields 3,513,793 word types against 3,513,777 for the unmodified `o200k` pattern — but compression is identical to three decimal places in all three languages. We report no measurable effect.

**Codepoint initialisation.** Byte-level BPE must spend merges reconstructing multi-byte codepoints before it can learn morphemes: every Telugu character shares a two-byte UTF-8 prefix that has to be merged first. We implemented a variant that seeds the merge table so every sufficiently frequent codepoint is a single token before frequency-driven merging begins. At a vocabulary of 48,000 this improved Telugu compression by 1.1% and nothing else. The explanation is visible in the proportion of learned tokens that are not valid UTF-8: 11.9% at a vocabulary of 2,000 but only 0.3% at 48,000. The problem codepoint initialisation addresses is a small-vocabulary artefact that has largely disappeared by the vocabulary sizes used in practice.

## 8 Limitations

We test one abugida. The mechanism predicts similar effects for Devanagari, Bengali, Tamil and Kannada, all of which write vowels as combining marks, but we have not measured them. We train two seeds per condition, which supports a range but not a variance estimate. Our models are 533M parameters trained on 9B tokens; whether the effect persists at larger scale is untested. The synthetic invoices used in the downstream evaluation have clean, consistent structure, whereas real scanned documents contain OCR noise and irregular layout, so those numbers should be read as optimistic. Finally, our Telugu supervised data is small (6,136 pairs) relative to English (130,234), and our Telugu test set is human-authored while some of the training data is not — a distribution mismatch that depresses Telugu QA scores independently of tokenisation.

## 9 Conclusion

A single character class in a pre-tokenisation regular expression determines whether Telugu words survive intact into byte-pair encoding. Restoring `\p{M}` to the letter class reduces Telugu token counts by 51.8% at identical vocabulary size and brings cross-language compression to near parity. In matched 533M-parameter models trained on identical token budgets, this improves held-out bits-per-byte by 5.5% for Telugu at a cost of about 1% for English and German, and improves Telugu extractive question answering by 45% relative in exact match.

The disparity between halving the token count and a 5.5% gain in bits-per-byte is the practical lesson. Any system serving Telugu through a `cl100k`-style tokeniser pays roughly twice the token cost for the same text, and fixing that is worth doing for the cost and context-length savings rather than for the perplexity. The correction is one character class.

## Reproducibility

Code, tokeniser model files, the synthetic invoice generator and all configuration files are available at <https://github.com/nikhilmule/combining-marks-matter>. Random seeds are fixed and reported for corpus sampling, MinHash permutations, data interleaving and model initialisation.

## References

- [1] Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems*, volume 36, pages 36963–36990, 2023.
- [2] Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David Mortensen, Noah A. Smith, and Yulia Tsvetkov. Do all languages cost the same? tokenization in the era of commercial language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9904–9923, Singapore, 2023. Association for Computational Linguistics.
- [3] OpenAI. tiktoken: a fast bpe tokeniser for use with openai’s models. <https://github.com/openai/tiktoken>, 2022.
- [4] Philip Gage. A new algorithm for data compression. *C Users Journal*, 12(2):23–38, 1994.

- [5] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, 2016. Association for Computational Linguistics.
- [6] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019.
- [7] Tomasz Limisiewicz, Jiří Balhar, and David Mareček. Tokenization impacts multilingual language modeling: Assessing vocabulary allocation and overlap across languages. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 4555–4570. Association for Computational Linguistics, 2023.
- [8] Mohammed Safi Ur Rahman Khan, Priyam Mehta, Ananth Sankar, Umashankar Kumaravelan, Sumanth Doddapaneni, Sparsh Suresh, Satish Golla, N. C. Gokul, Avik Bhattacharyya, Mitesh M. Khapra, and Anoop Kunchukuttan. Indicllmsuite: A blueprint for creating pre-training and fine-tuning datasets for indian languages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [9] Abhishek Kumar Singh, Vishwajeet Kumar, Rudra Murthy, Jaydeep Sen, Ashish Mittal, and Ganesh Ramakrishnan. Indic QA benchmark: A multilingual benchmark to evaluate question answering capability of LLMs for indic languages. arXiv:2407.13522, 2024.
- [10] Divyanshu Kakwani, Anoop Kunchukuttan, Satish Golla, N. C. Gokul, Avik Bhattacharyya, Mitesh M. Khapra, and Pratyush Kumar. Indicnlp suite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for indian languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4948–4961. Association for Computational Linguistics, 2020.
- [11] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems*, volume 37, pages 30811–30849, 2024.
- [12] Guilherme Penedo, Hynek Kydlíček, Vinko Sabolčec, Bettina Messmer, Negar Foroutan, Amir Hossein Kargaran, Colin Raffel, Martin Jaggi, Leandro Von Werra, and Thomas Wolf. Fineweb2: One pipeline to scale them all – adapting pre-training data processing to every language, 2025.
- [13] Andrej Karpathy. nanogpt. <https://github.com/karpathy/nanoGPT>, 2022.
- [14] Andrej Karpathy. nanochat. <https://github.com/karpathy/nanochat>, 2025.
- [15] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030, 2022.
- [16] Thuat Nguyen, Chien Van Nguyen, Viet Dac Lai, Hieu Man, Nghia Trung Ngo, Franck Dernoncourt, Ryan A. Rossi, and Thien Huu Nguyen. Culturax: A cleaned, enormous, and multilingual dataset for large language models in 167 languages. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, 2024.
- [17] Sumanth Doddapaneni, Rahul Aralikatte, Gowtham Ramesh, Shreya Goyal, Mitesh M. Khapra, Anoop Kunchukuttan, and Pratyush Kumar. Towards leaving no indic language behind: Building monolingual corpora, benchmark and models for indic languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.
- [18] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for SQuAD. *arXiv preprint arXiv:1806.03822*, 2018.
- [19] Timo Möller, Julian Risch, and Malte Pietsch. Germanquad and germandpr: Improving non-english question answering and passage retrieval. In *Proceedings of the 3rd Workshop on Machine Reading for Question Answering*, pages 42–50. Association for Computational Linguistics, 2021.
- [20] Jonathan H. Clark, Eunsol Choi, Michael Collins, Dan Garrette, Tom Kwiatkowski, Vitaly Nikolaev, and Jennimaria Palomaki. Tydi qa: A benchmark for information-seeking question answering in typologically diverse languages. *Transactions of the Association for Computational Linguistics*, 8:454–470, 2020.